

Topic : Accurate Real-Time Stereo Matching on the GPU for 3D Reconstruction

Summer semester 2014

Master Project

Babak Ebrahimi

babak.ebrahimi1987@gmail.com

Supervisors: Dr. Norbert Schmitz
and Dipl. Vladislav Golyanik

- Introduction
- Related works
- Work Contribution
- Matching algorithm
- Implementation on GPU
- Results
- Conclusion and feature work

- The main focus of this Project is on the following paper:

Accurate Real-Time Multi-Camera Stereo-Matching on the GPU for 3D Reconstruction

Klaus Denker
HTWG Konstanz, Germany
kdenker@htwg-konstanz.de

Georg Umlauf
HTWG Konstanz, Germany
umlauf@htwg-konstanz.de

ABSTRACT

Using multi-camera matching techniques for 3d reconstruction there is usually the trade-off between the quality of the computed depth map and the speed of the computations. Whereas high quality matching methods take several seconds to several minutes to compute a depth map for one set of images, real-time methods achieve only low quality results. In this paper we present a multi-camera matching method that runs in real-time and yields high resolution depth maps.

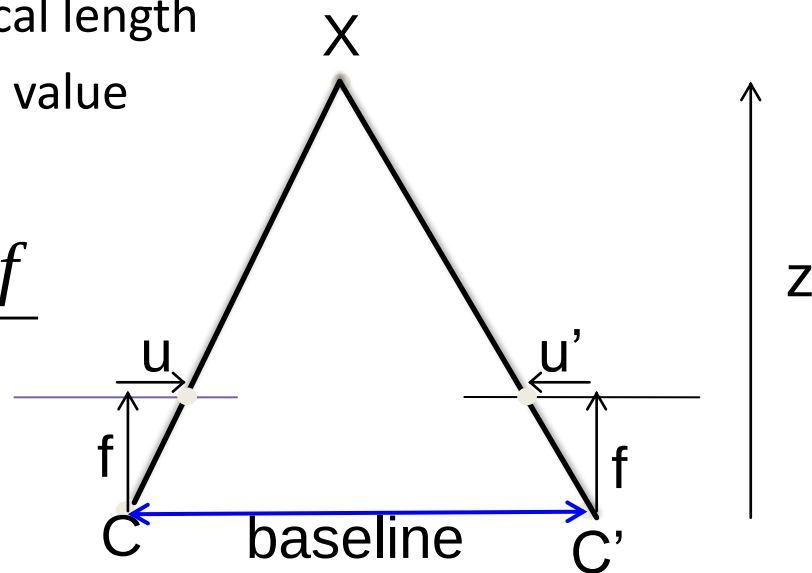
Our method is based on a novel multi-level combination of normalized cross correlation, deformed matching windows based on the multi-level depth map information, and sub-pixel precise disparity maps. The whole process is implemented completely on the GPU. With this approach we can process four 0.7 megapixel images in 129 milliseconds to a full resolution 3d depth map. Our technique is tailored for the recognition of non-technical shapes, because our target application is face recognition.

Keywords

Stereo-matching, multi-camera, real-time, gpu, computer vision.

- **Stereo matching** is a technique to compute depth information of a captured object or environment from two or more 2d camera images.
- By having baseline distance, camera's focal length and the disparity we can compute depth value of the pixels:

$$disparity = u - u' = \frac{baseline \times f}{z}$$



- **Main challenge** of stereo matching: the **tradeoff** between the **quality** and the **computation time** of a depth map.
- There are two main group of methods for Stereo Matching :
 1. **High quality** Methods
 2. **GPU-Based** Methods

- **High quality** methods **disadvantage**:
need at least several seconds to compute a single depth map from one set of images
- For application **like robotics** faster computation time is more important than the quality of depth map which led to development of GPU-based real-time matching methods.
- Our target application here is **3d Face Recognition** that whole reconstruction and recognition needs to be done in **real-time** (less than half a second).

- High quality Methods
 1. Adaptive least squares correlation [3]
 2. Region growing algorithms, e.g. [4, 5]
 3. Phase correlation method in [6]
 4. Global optimization of a Markov Random Field (MRF) [8].
 5. Particle cloud optimization used by [9]

- GPU Methods

1. A typical approach of GPU methods which uses sum of squared differences or sum of absolute differences for matching score [10].

Advantage and disadvantage : It is fast but yields low quality disparity maps.

1. An approach which five calibrated cameras are matched at once. In other configuration a reconfigurable array of 48 cameras were used [11,13].

Advantage and disadvantage: using one pixel for matching computation on the GPU which makes it faster but unstable.

3. A plane sweep method for depth estimation on a large number of non-rectified images[12].
Advantage: includes a volumetric reconstruction of the objects
Disadvantage: It is slower than other GPU methods.

4. A method which uses the pyramidal matching kernel and mip-mapping and adds a foreground/ background separation on the GPU[14].
advantage: avoids typical artifacts of the pyramidal kernel like wrong depth estimates for regions with low texture details usually found in the background.

- Contribution of this work:
 1. The GPU optimized use of weighted normalized cross-correlations
 2. Using shared memory for increasing the speed of depth map computation
- To compute the depth maps in real-time the process is implemented on the GPU.

Initial inputs of the program:



Image Source : "NVIDIA CUDA Programming Guide" version 1.1

1- Weighted average color:

$$\bar{f}_i = \sum_{(x,y) \in W_i} w(x,y) f(x,y)$$

2- Weight function:

$$w(x,y) = \cos^2(\pi x/a) \cdot \cos^2(\pi y/a)$$

3- Weighted auto-correlation: $\alpha_i = \sum_{x,y} w(x,y) [f_i(x,y) - \bar{f}_i]^2$

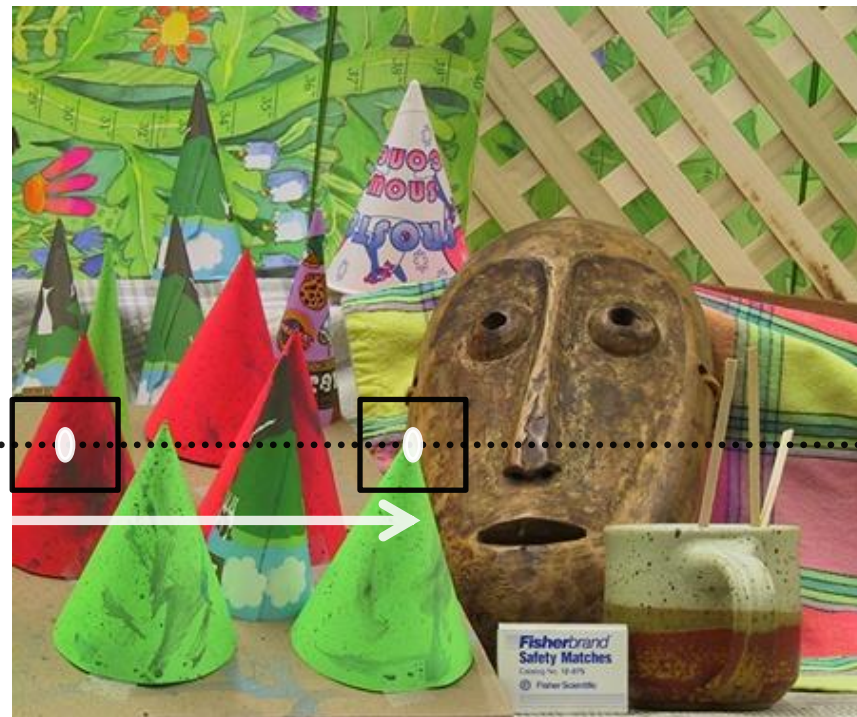
4- Similarity of two matching Windows computed by weighted cross correlation:

$$\beta_{i,j} = \sum_{x,y} w(x,y) [f_i(x,y) - \bar{f}_i] \cdot [f_j(x,y) - \bar{f}_j]$$

5- Weighted normalized score:

$$\gamma_{i,j} = \beta_{i,j} / \sqrt{\alpha_i \cdot \alpha_j}$$

Matching Window Movement

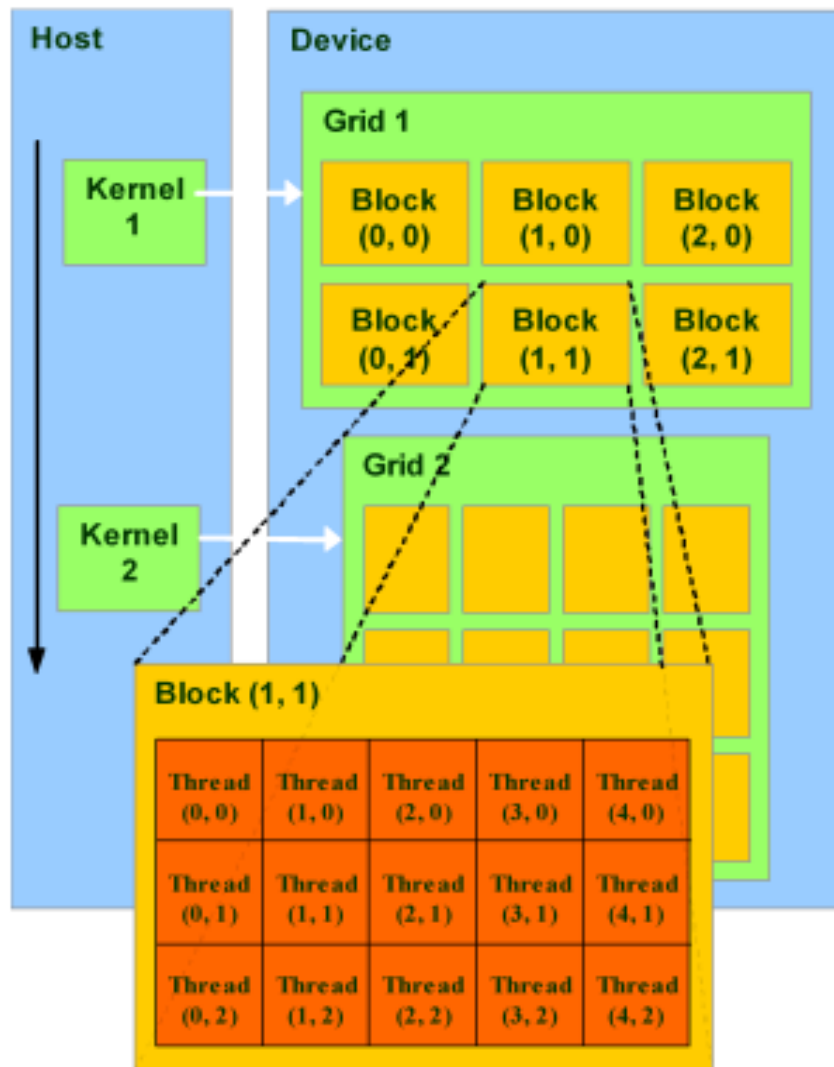


Implementation on GPU

- Kernel invocation:
- `kernelName<<<gridSize, blockSize>>>(...)`
- where:
- `gridSize`: number of blocks in the grid
- `blockSize`: number of threads in each block



Image divided by Grid blocks



Implementation on GPU

- GPU memory structure

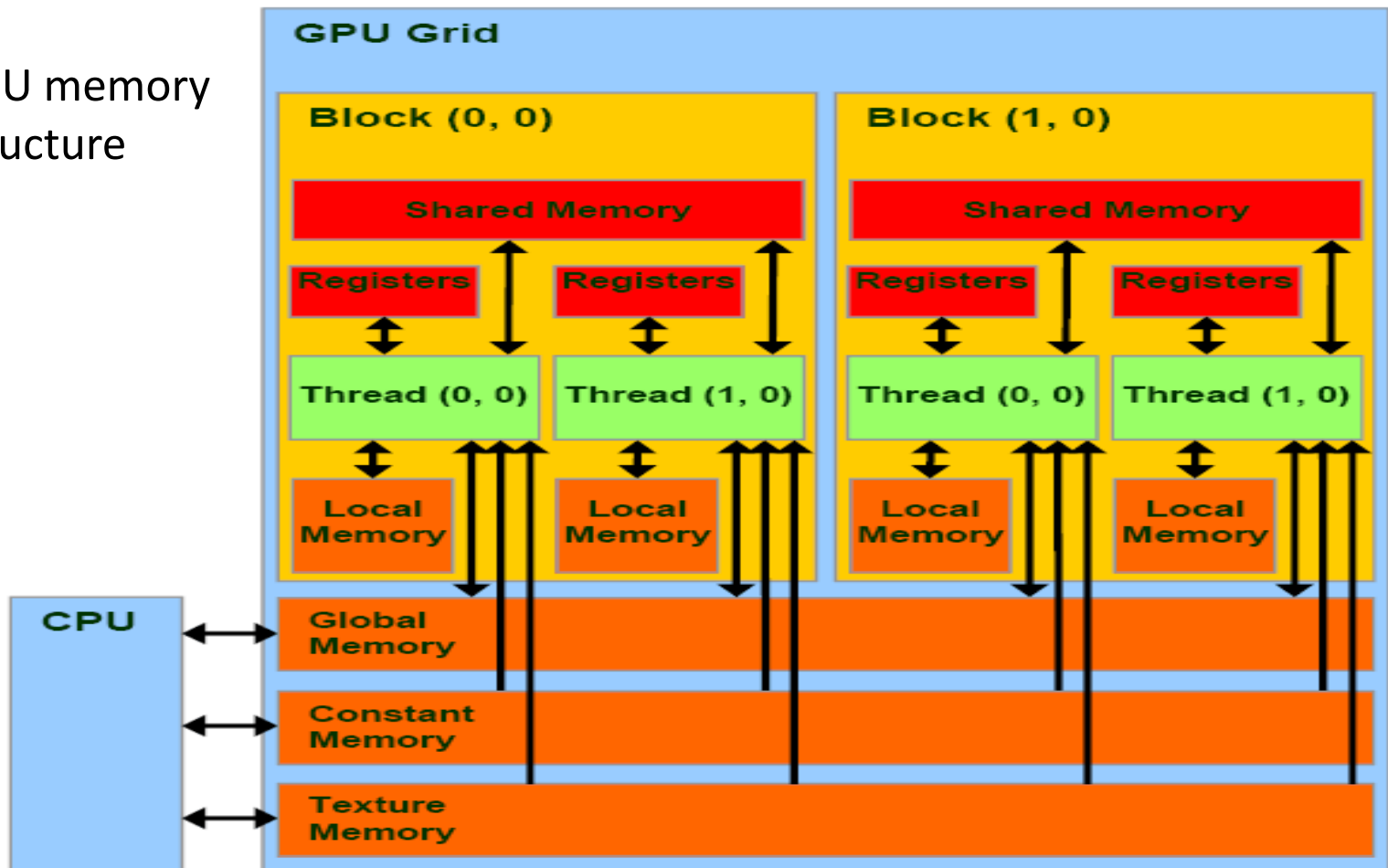


Image Source : "NVIDIA CUDA Programming Guide" version 1.1

- [illegible]

15

- Transferring to shared memory
- Number of threads in each block(32×32) is not enough for transferring all the pixel's values so we have to call some of the threads two times

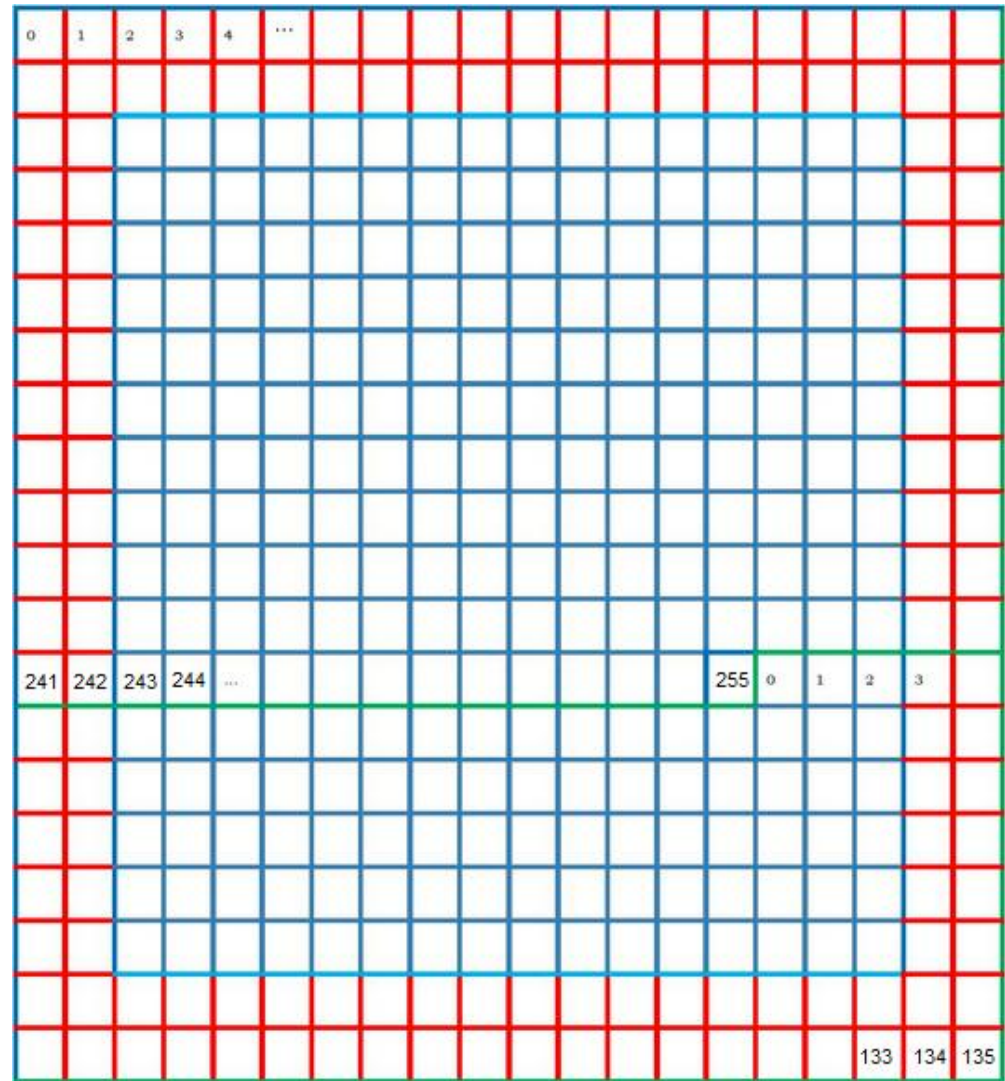
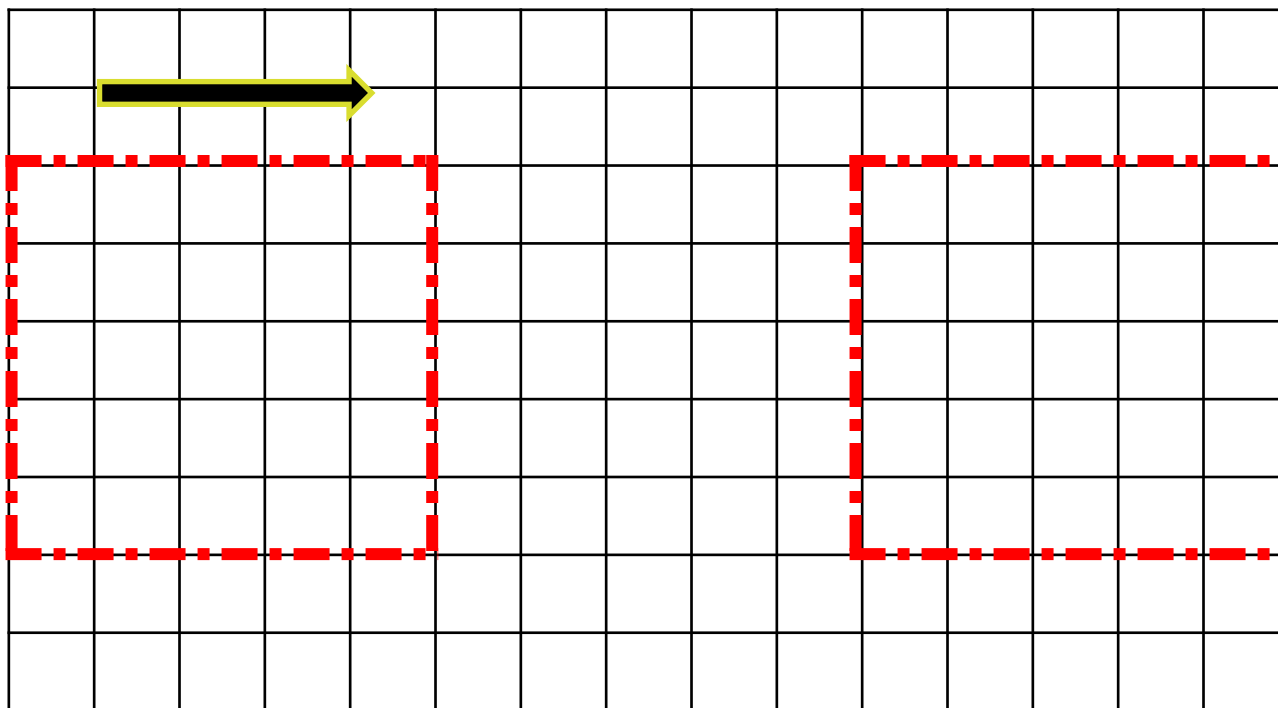
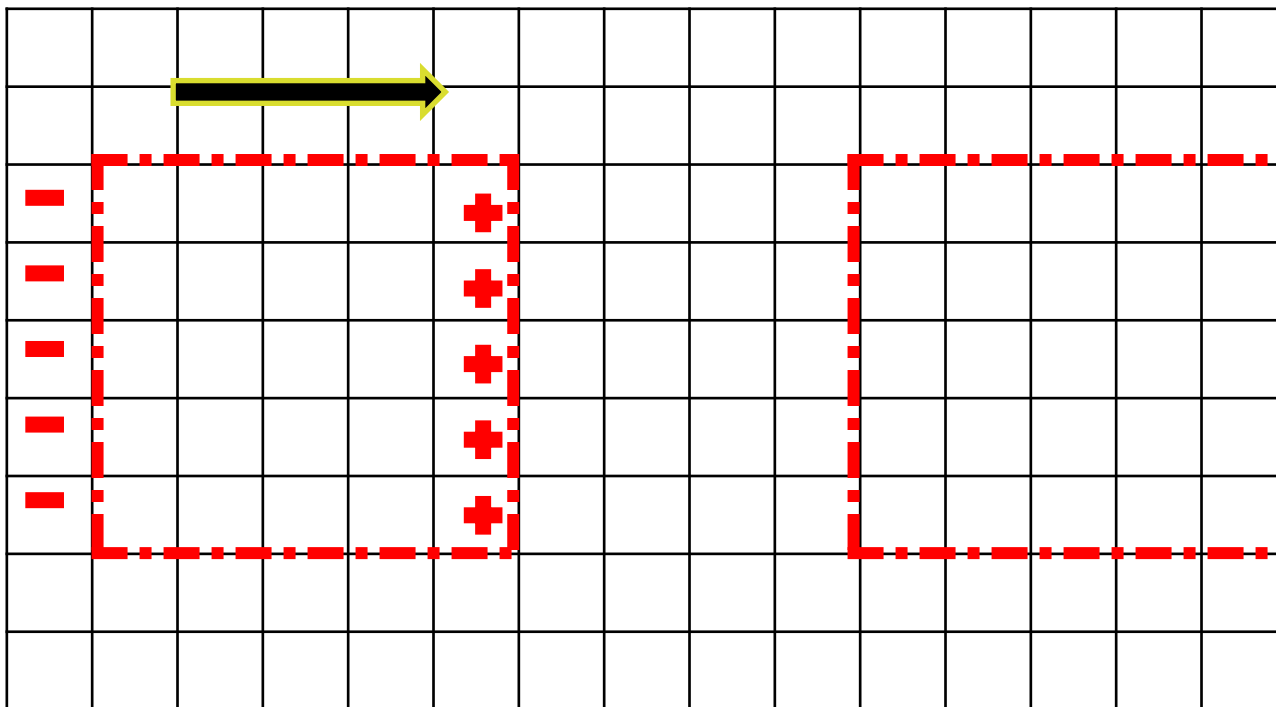


Image Source : www.stackoverflow.com

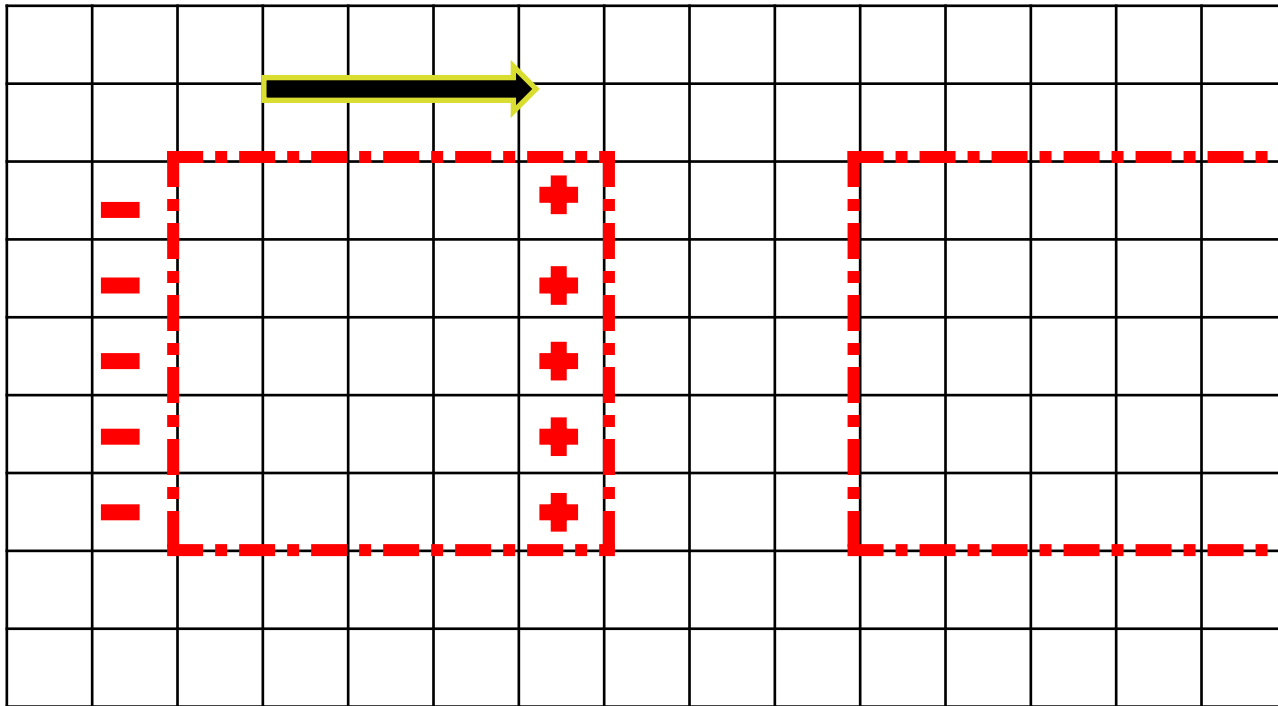
- First transfer both blocks to shared memory and compute Weighted average color and Weighted auto-correlation in kernel 1 .
- Then in kernel 2 transfer both blocks to shared memory and compute weighted cross correlation by moving the second matching window.



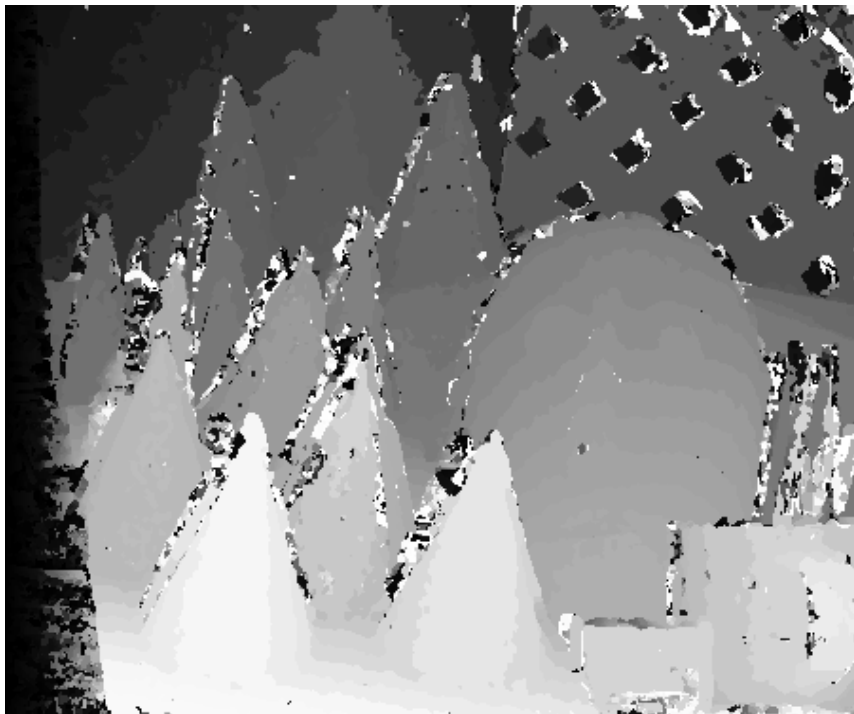
- If the threads were in block border add one column to the right side of block in shared memory from the global memory and remove one column from the left side of block in the shared memory.



- If the threads are not in the border just transfer its value to the left pixel value.



The results on CPU and GPU(GeForce GTX 770) :



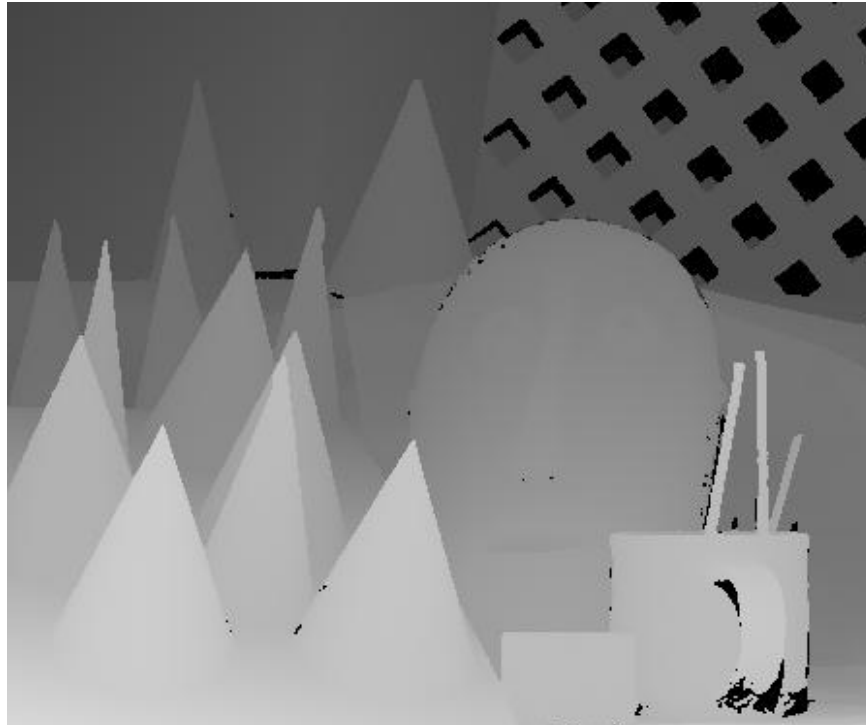
CPU output



GPU output

Image Source : "NVIDIA CUDA Programming Guide" version 1.1

The Ground Truth:



Ground Truth

- We implemented the same method on CPU for comparing the results with GPU.

Device	Average time
CPU	30.1 seconds
GPU(GeForce GTX 770 cuda)	48.9 ms

- Around 600 times faster computation in GPU function over CPU function
- Only difference between two implementation is that I saved the values of weighted cross correlation and Weighted normalized score in an array and then found the maximum but in GPU I didn't save these values anywhere

- According to Nvidia Visual Profiler:

GPU implementation utilization	50%
Shared memory usage per block for each kernel	2.82 KB
Achieved occupancy for First Kernel	99.5%
Theoretical occupancy for First Kernel	100%
Achieved occupancy for Second Kernel	50%
Theoretical occupancy for Second Kernel	50%

- Using GPU registers was the limiter for second kernel occupancy.
- 98.8 % of the computation time was for second kernel and 1.1% of the computation time was for first the kernel.

- **Conclusion:**
- The speed of GPU was very satisfying
- The quality of depth map in this approach needs some improvement to be comparable with state of the art of stereo matching benchmarks
- **Future possible work:**
- Adding more cameras for improving the results
- recording synchronous video and use the matching information of an earlier video frame to improve the performance.

Question



1. Klaus Denker, Georg Umlauf: **Accurate Real-Time Multi-Camera Stereo-Matching on the GPU for 3D Reconstruction**. Journal of WSCG 19(1): 9-16 (2011)
2. Jens Hensler, Klaus Denker, Matthias O. Franz, Georg Umlauf: **Hybrid Face Recognition Based on Real-Time Multi-camera Stereo-Matching**. ISVC (2) 2011: 158-167
3. A W Gruen. Adaptive least squares correlation: A powerful image matching technique. South African Journal of Photogrammetry, Remote Sensing and Cartography, 14:175–187, 1985
4. Yasutaka Furukawa and Jean Ponce. Accurate, dense, and robust multi-view stereopsis. In CVPR, pages 1–8, 2007.
5. Andreas Klaus, Mario Sormann, and Konrad Karner. Segmentbased stereo matching using belief propagation and a selfadapting dissimilarity measure. In Proc. of the 18th Int. Conf. on Pattern Recognition, pages 15–18, 2006
6. T. Shibahara, T. Aoki, H. Nakajima, and K. Kobayashi. A subpixel stereo correspondence technique based on 1d phase-only correlation. In ICIP07, pages 221–224, 2007
7. Kenneth Moreland and Edward Angel. The FFT on a GPU. In Proc. of the ACM Conf. on Graphics Hardware, pages 112–119, 2003

8. Neill D. Campbell, George Vogiatzis, Carlos Hernández, and Roberto Cipolla. Using multiple hypotheses to improve depthmaps for multi-view stereo. In Proc. of the 10th European Conf. on Computer Vision, pages 766–779, 2008
9. Alexander Hornung and Leif Kobbelt. Interactive pixelaccurate free viewpoint rendering from images with silhouette aware sampling. Comp. Graph. Forum, 28(8):2090–2103, 2009
10. Ruigang Yang and Marc Pollefeys. A versatile stereo implementation on commodity graphics hardware. Real-Time Imaging, 11(1):7–18, 2005
11. Ruigang Yang, Greg Welch, and Gary Bishop. Real-time consensus-based scene reconstruction using commodity graphics hardware. In Proc. of the 10th Pacific Conf. on Computer Graphics and Applications, pages 225–235, 2002
12. Christopher Zach, Mario Sormann, and Konrad F. Karner. High-performance multi-view reconstruction. In 3DPVT, pages 113–120, 2006
13. Cha Zhang and Tsuhan Chen. A self-reconfigurable camera array. In SIGGRAPH '04: ACM SIGGRAPH 2004 Sketches, page 151, 2004
14. Jia-Ching Cheng and Shin-Jang Feng. A real-time multiresolutional stereo matching algorithm. In ICIP (3), pages 373–376, 2005.
15. F. Devernay. A non-maxima suppression method for edge detection with sub-pixel accuracy. Technical Report RR-2724, INRIA, 1995.